

Rapport ATM

Github: <https://github.com/gqysmart/cppcourse-bankATM.git>

Description des actions réalisées :

- examen du code source existant (`src/App.cpp`) pour confirmer la logique de vérification du code PIN ;
- exécution automatisée du binaire `build/ATM` pour observer le comportement en cas de succès et d'échec ;
- préparation de captures d'écran représentatives et génération de ce rapport.

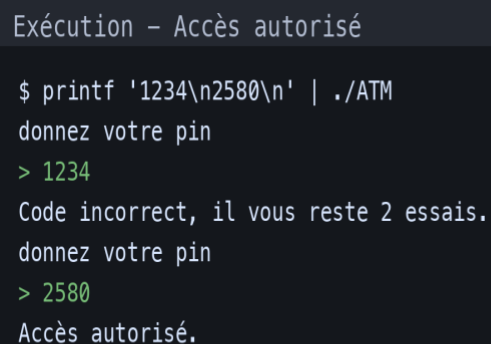
Algorithme de vérification du code PIN :

1. définir le code secret attendu `password = "2580"` ;
2. pour chaque tentative (maximum trois) : demander la saisie du PIN, comparer la valeur au mot de passe et afficher le résultat ;
3. si toutes les tentatives sont erronées, l'application se termine sans autoriser l'accès.

Remarques : le programme repose sur des entrées console standards et n'enregistre pas d'état persistant. Toute nouvelle demande exige de relancer l'application.

Captures d'écran illustrant deux scénarios :

Scénario 1 : saisie correcte au deuxième essai.



```
Exécution - Accès autorisé
$ printf '1234\n2580\n' | ./ATM
donnez votre pin
> 1234
Code incorrect, il vous reste 2 essais.
donnez votre pin
> 2580
Accès autorisé.
```

Scénario 2 : trois codes erronés consécutifs.

Exécution – Accès refusé

```
$ printf '0000\n1111\n2222\n' | ./ATM  
donnez votre pin  
> 0000  
Code incorrect, il vous reste 2 essais.  
donnez votre pin  
> 1111  
Code incorrect, il vous reste 1 essais.  
donnez votre pin  
> 2222  
Code incorrect, il vous reste 0 essais.
```